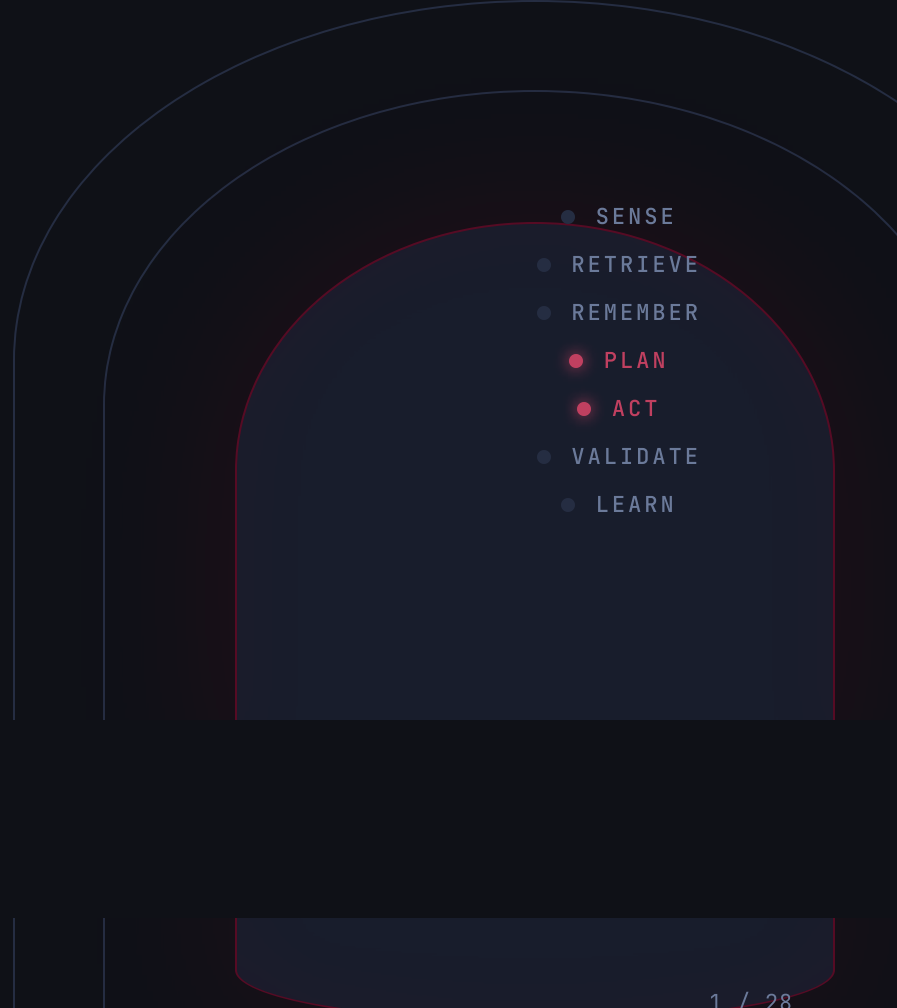


The model is the engine. The **harness** is the car.

What makes a good agentic coding harness, and how clew keeps it coherent.

- 
- A diagram illustrating the components of an agentic coding harness. It features three concentric, rounded rectangular outlines. The innermost outline is red and contains a vertical list of seven items, each preceded by a small circle. The middle and outer outlines are light blue. The items in the list are: SENSE, RETRIEVE, REMEMBER, PLAN, ACT, VALIDATE, and LEARN. The items PLAN and ACT are highlighted with red circles and red text, while the others have grey circles and grey text.
- SENSE
 - RETRIEVE
 - REMEMBER
 - **PLAN**
 - **ACT**
 - VALIDATE
 - LEARN

SECTION 01 / 05

The Problem

Agents can't remember between sessions, and every obvious fix for it fails.

01

Most harnesses only have the shallowest memory, yet memory is the moat.

Working memory

the active task: recent files, notes, distilled
compacted into the prompt each turn · lost at session end

Episodic memory

full transcript of this session: every request, tool call, result
saved to a file → `/resume` · but locked to this one thread

Long-term memory

AGENTS.md · CLAUDE.md · the artefact store
plain files · reloaded at the start of every session

← most stop here

the moat

2 of 3 session-bound

Working memory is distilled, then discarded at session end; episodic is a resumable transcript but locked to its own thread. Only long-term (plain files) crosses into the next session.

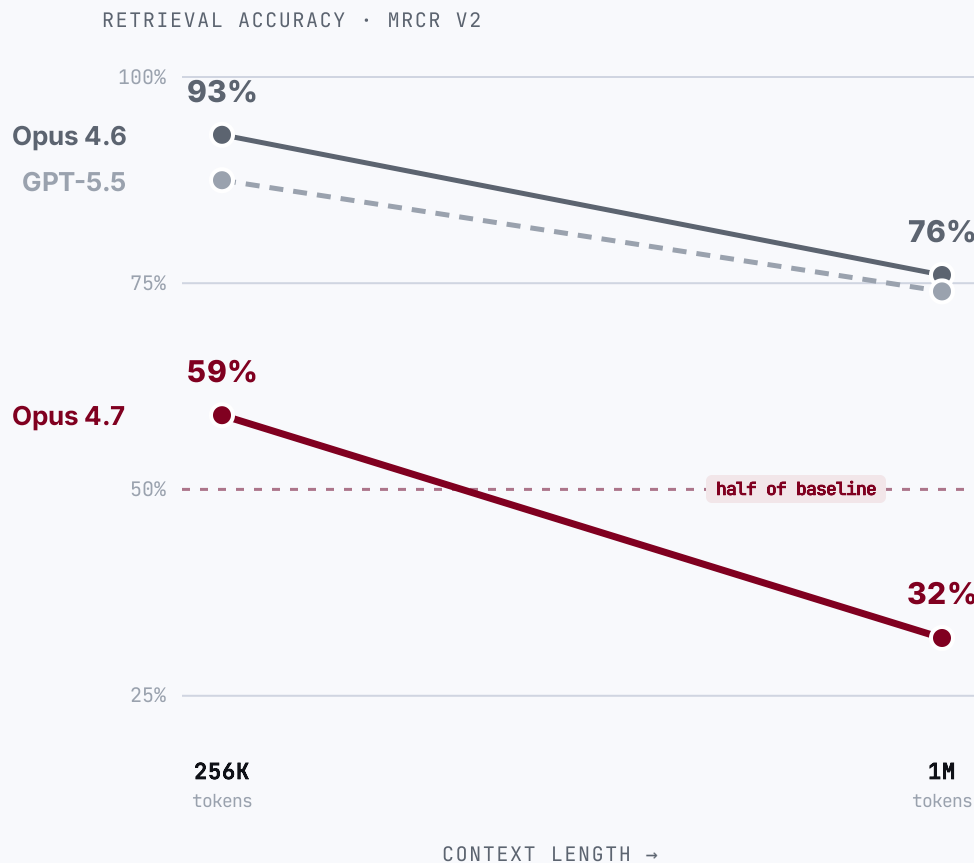
0 tokens / fetch

And it's a local file read, no MCP API roundtrip, no token cost, reloaded at the start of every session.

THE OBVIOUS OBJECTION "A frontier window holds a million tokens now, so why not just let the *window* be the memory?" ANSWERED NEXT →

No. A bigger context window is not bigger memory.

Bury one fact in a million-token prompt and ask for it back: the frontier model returns it barely **1 in 3** times. The window **forgets** as you fill it.



The advertised window is **not the usable window.**

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context, a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens. Context rot isn't new (NoLiMa, 2025).

Writing it down isn't memory either.

OpenAI shipped a 1M-line product with **zero** hand-written code, then found the obvious fix, one big context file, fails.

1M lines of code

0 hand-written

1,500 PRs merged

“One big **AGENTS.md**” failed four ways

- 📦 **Crowds out the task.** Context is scarce: the giant file pushes out the code and the docs.
- 🔊× **Too much guidance becomes none.** When everything is “important,” nothing is.
- 📄 **It rots instantly.** A graveyard of stale rules: the agent can't tell what's still true.
- 🔍× **It can't be verified.** Drift is inevitable without mechanical checks.

If it's not in-context, it doesn't exist

Anything in Slack, Google Docs, or someone's head is invisible to the agent.

OpenAI's fix

A **~100-line table of contents** pointing into a **structured docs/ tree**: a map, not an encyclopedia.

2025 was the “Year of the Agent.” It didn't land.

The hype was model-led, but in production, the model alone fell short, expensively.

95%

of enterprise GenAI pilots showed **no measurable P&L impact**

MIT studied 300 deployments + 350 leaders. The blocker wasn't model IQ; it was integrating AI into real workflows.

~59%

is the **best** frontier model's score on realistic enterprise tasks

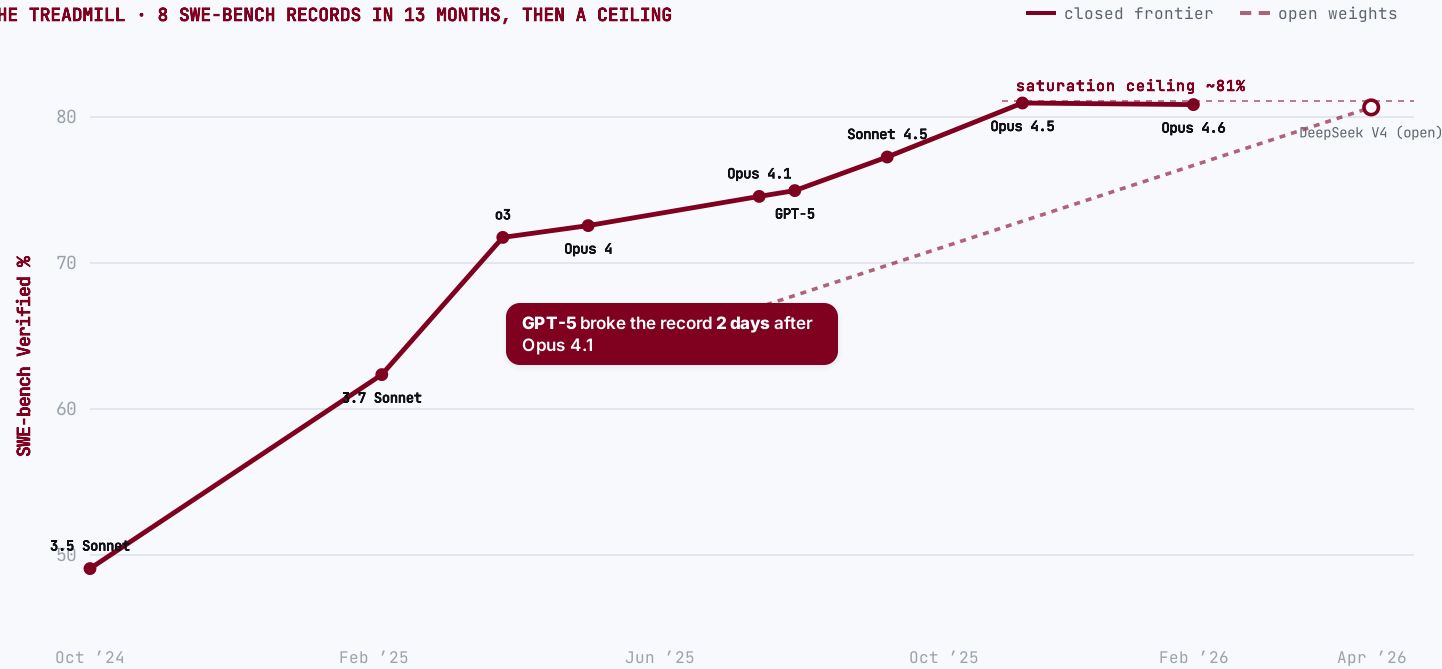
Scale's SWE-bench Pro: 1,865 real tasks across 41 professional repos (2026). The same models score 80–95% on the benchmark everyone quotes; the gap is the reality.

The missing variable wasn't intelligence; it was the **harness around the model**.

Models leapfrog every benchmark, then commoditize.

A new frontier model tops every benchmark... until the next does, **days** later. Then it saturates ~81%, open weights reach **parity**, and the benchmark itself gets retired.

THE TREADMILL • 8 SWE-BENCH RECORDS IN 13 MONTHS, THEN A CEILING



🔗 Open weights, now at parity

≈ 80.6% = the frontier

DeepSeek V4-Pro matches the closed SOTA on SWE-bench Verified (Apr '26). Composite gap now ~4 months / 8 ECI pts (Epoch AI, May '26).

⚠️ The benchmark itself broke

59% of hard tests flawed

OpenAI retired SWE-bench Verified (Feb '26): frontier models had trained on the answers; it's saturated. → SWE-bench Pro.

📉 Inference is collapsing

≈ 280× cheaper / 18 mo

GPT-3.5-level tokens: \$20.00 → \$0.07 per M. ~10× every year: "LLMflation."

"The models are getting commoditized."

Satya Nadella • Microsoft CEO • B22 Pod, Dec 2024



The model is a commodity, not a moat.

Whatever tops the leaderboard this week is matched, undercut and open-weighted within months, and the benchmark itself gets retired.

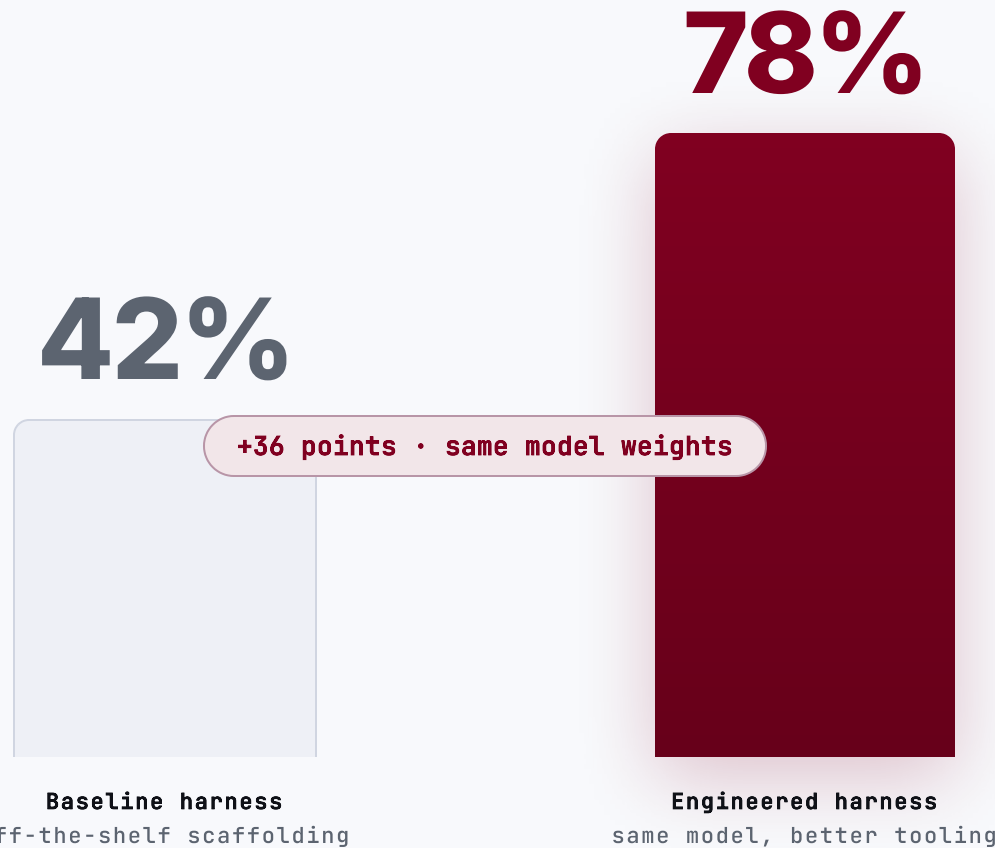
SECTION 02 / 05

The Agentic **Harness**

Why the tooling around the model, not the model, is the performance lever.

02

Hold the model fixed, swap the harness: 42% → 78%.

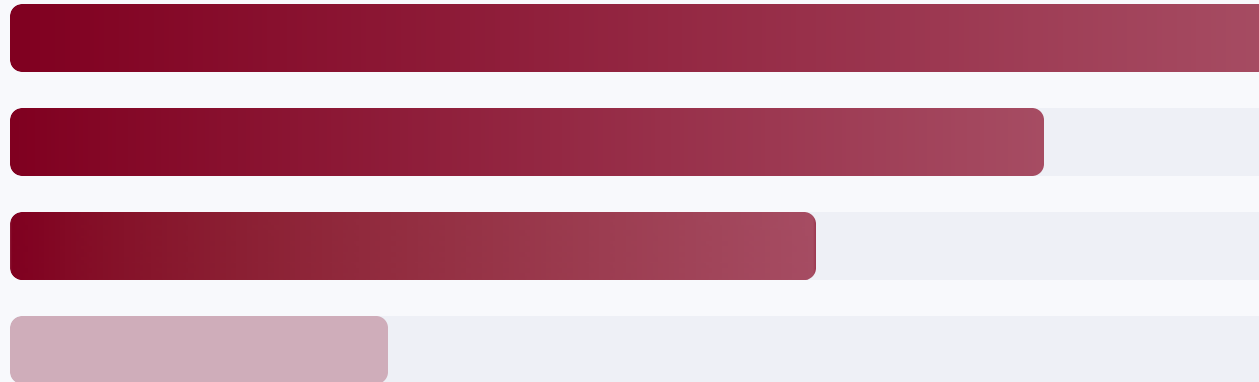


Claude Opus 4.5 • HAL CORE-Bench • the only thing that changed was the harness

Performance lives in the tooling around the model, not the model.

Ablation studies rank what actually moves agent performance. Model prompt-tuning sits **last**.

- 1 **Tools**
- 2 **Control layer**
context · execution · recovery (middleware)
- 3 **Long-term memory**
- 4 **System prompt**



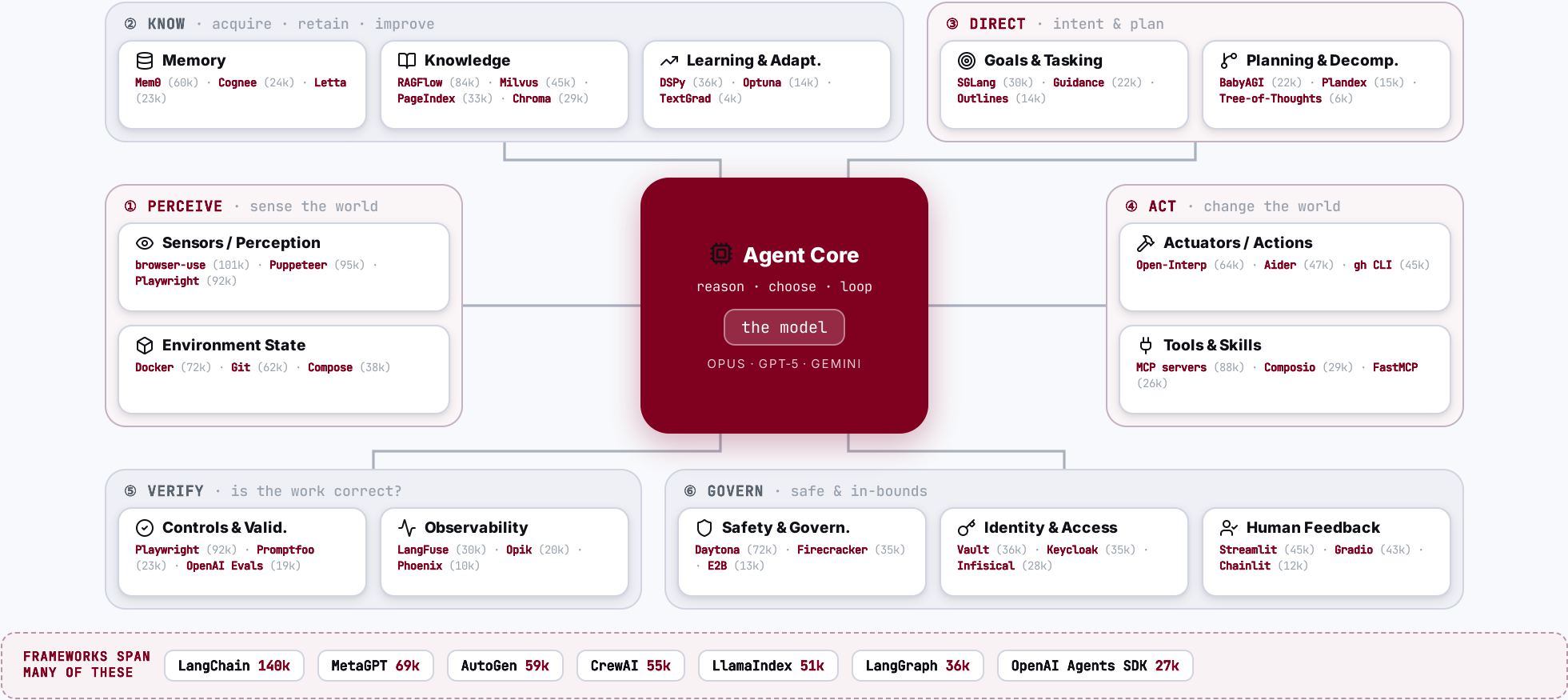
THE INVERSION

Most teams pour effort into #4, the prompt. The gains sit in #1–3.

THE MODEL

Table stakes once it is capable.
The harness is the differentiator.

The agent core, wrapped in 14 capabilities, and who leads each.



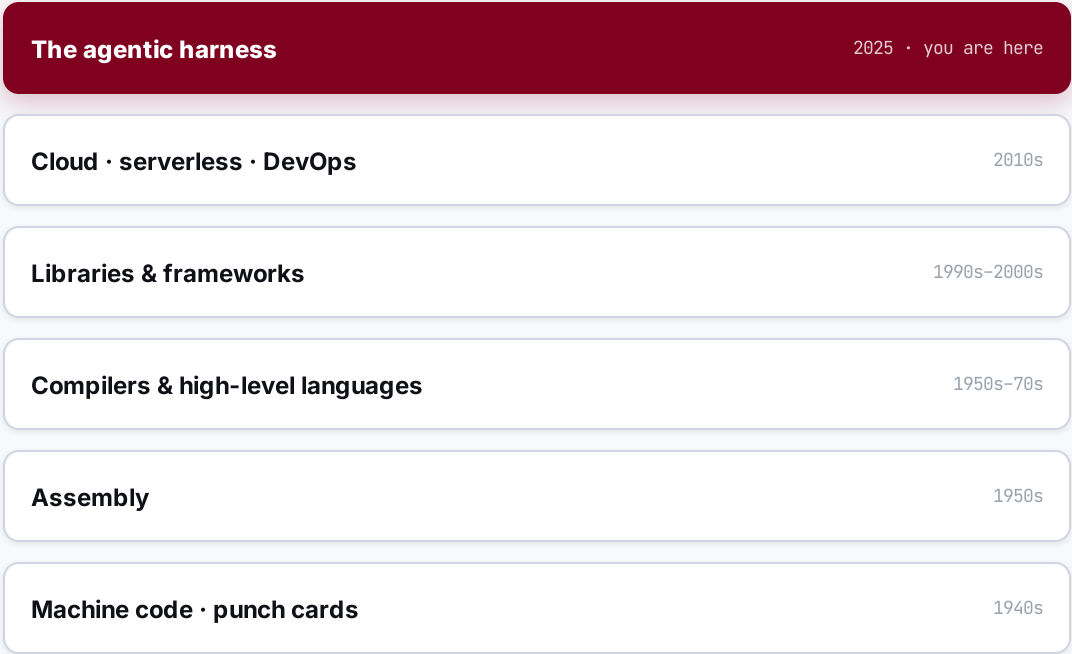
Do you really think software engineering is dead?

That map was **50 tools** wrapping a single model: each its own system, all to be wired together and kept alive.

50 tools around one model
and only the top few per capability; 80+ ship in the wild

-  **Each tool is its own system**
its own API, config, failure modes and learning curve
-  **Orchestration is the hard part**
wiring 50 moving parts into one reliable, debuggable loop
-  **Maintenance never stops**
versions drift, tools break, the harness needs constant upkeep

THE LADDER OF ABSTRACTION • every rung was once “the end of programming”



No. We’ve just climbed another rung on the ladder of abstraction.

Software engineering didn’t disappear; building and maintaining this harness is the engineering now.

BUILDING WITH AGENTS • IN PRACTICE

Dev Tooling

The editor, model, agent-CLI and IDE you build with, open-source or proprietary, and how engineers climb from vibe-coding to real agentic engineering. The harness and the method underneath stay the same.



Open-source or proprietary: **your setup, your call.**

Three tiers: bring a model, compose a custom harness, or run a full IDE, each available **fully open** or **fully proprietary**. What doesn't change is the method underneath.

	 OPEN-SOURCE · self-hostable	 PROPRIETARY · hosted
 Full IDE ALL-IN-ONE	VS Code +187k Zed +86k Continue +35k Tabby +34k Zed standalone · VS Code + ext	Cursor Windsurf Antigravity Kiro closed-source · bundled
 Custom harness EDITOR + AGENT	opencode +180k Cline +64k Goose +50k Aider +47k Crush +26k open + model-agnostic	Claude Code +135k Codex +94k Copilot Antigravity CLI Amp proprietary · vendor-hosted
 Inference & routing OPTIONAL	Ollama +175k llama.cpp +118k vLLM +85k LiteLLM +52k self-host · open	OpenRouter Groq Hugging Face Bedrock Vertex AI Azure OpenAI managed · cloud gateway
 Bring your own model THE MODEL	DeepSeek Llama Qwen gpt-oss Mistral open weights · self-hostable	Claude GPT-5 Gemini 3 Grok frontier · hosted

 **The method**
THE CONSTANT, EITHER SIDE

spec

 →

plan

 →

grill-me

 →

peer-review

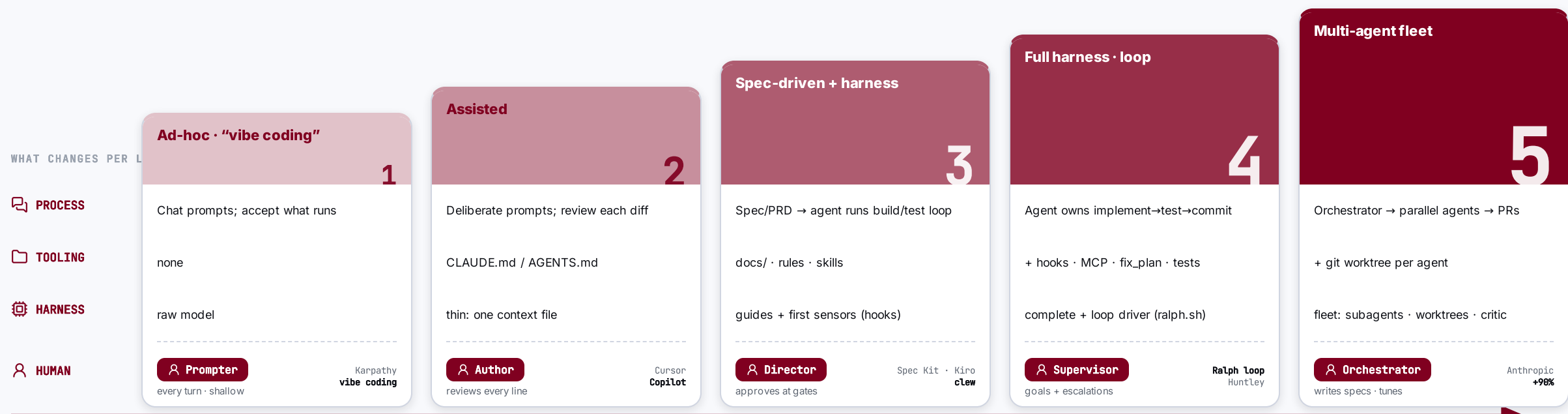
 →

loop

· artefacts · rules · skills · the metamodel

From “vibe coding” to *agentic engineering*: five levels.

Autonomy climbs and the harness thickens, but the human moves **up** the stack (prompter → orchestrator), never out of it.



INCREASING AUTONOMY · DEEPER HARNESS → role rises with every step ↗, never leaves the loop

⚠ **Even L5 is *supervised* autonomy, not hands-off.** METR: experienced devs were 19% *slower* on mature codebases; SWE-bench Pro tops out ~23%; Ralph’s author: “no way I’d use it on an existing codebase.” The guardrail against Thoughtworks’ “**cognitive debt**” is keeping humans on standards, architecture & final sign-off.

SECTION 03 / 05

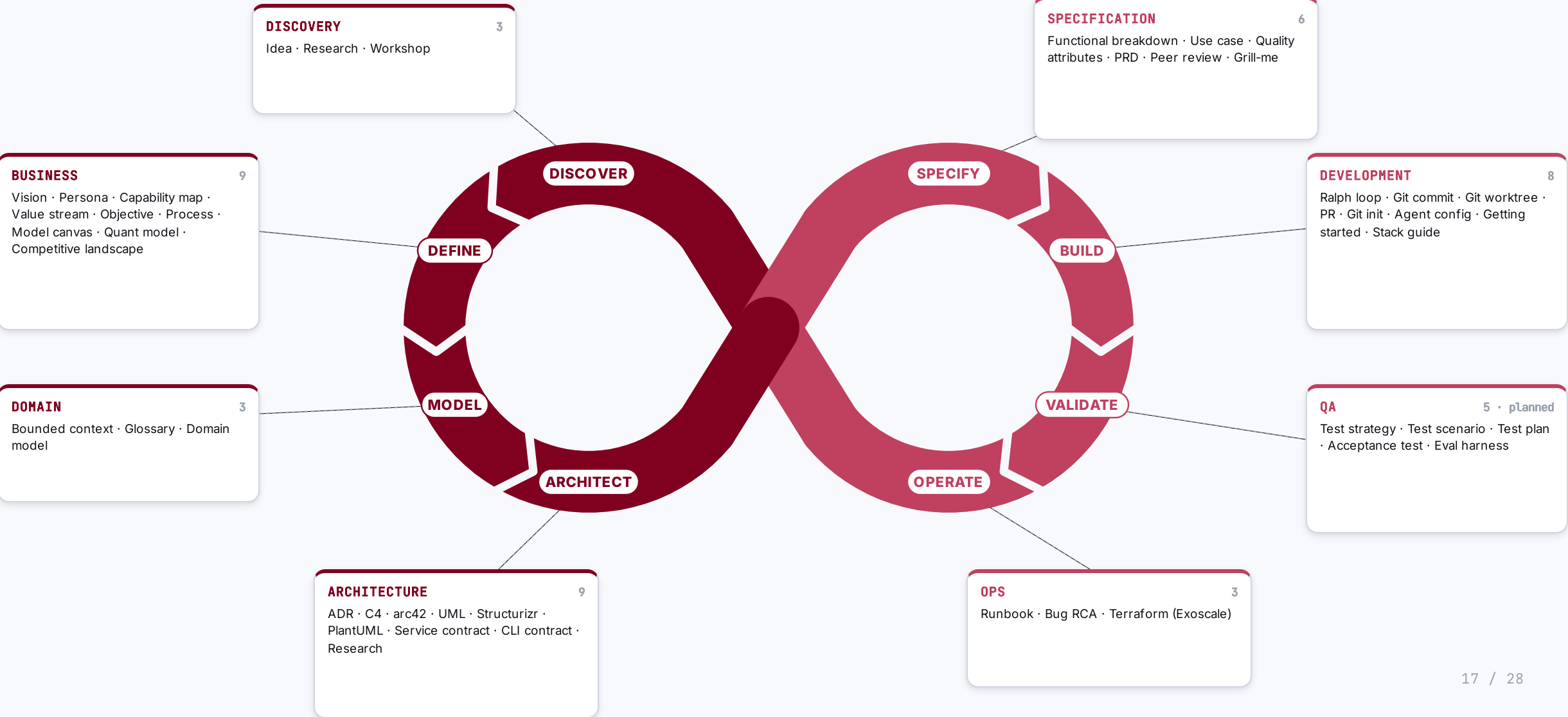
The Metamodel

How an agent armed with skills and rules turns business context into deployable artefacts, every step traceable.

03

Skills & rules wrap the whole lifecycle, from discovery to operations.

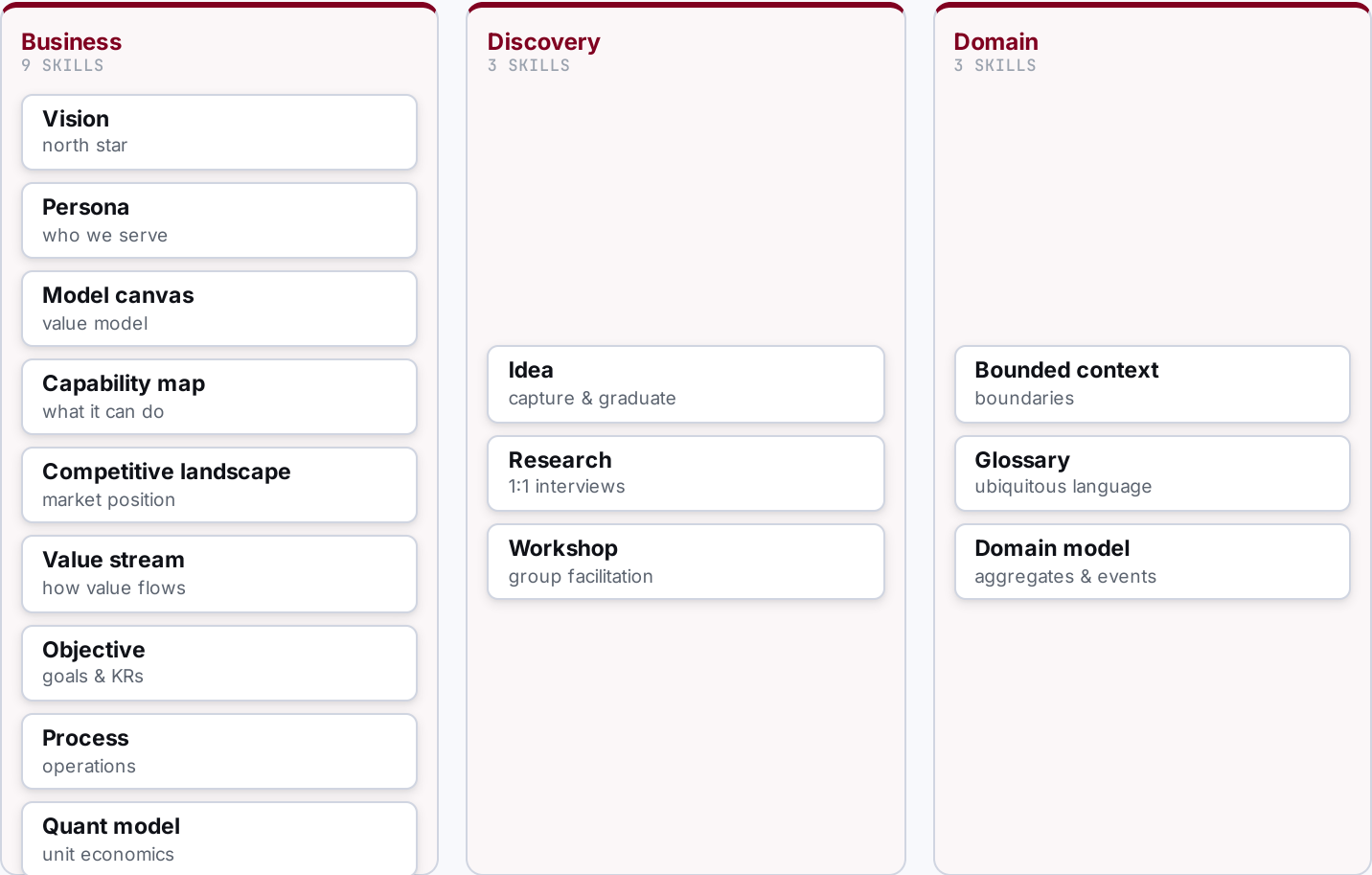
Far beyond DevOps: eight phases from business discovery to operations, each owned by a prefix, all tracing back to one metamodel.



Discover & Define • make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Brainstorm & capture**
ideas, hypotheses
- **Research & scan**
market, competitors
- **Interview & workshop**
with the business
- **Model the business**
vision → objectives
- **Define the domain**
ubiquitous language



Architect the solution · turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

- 

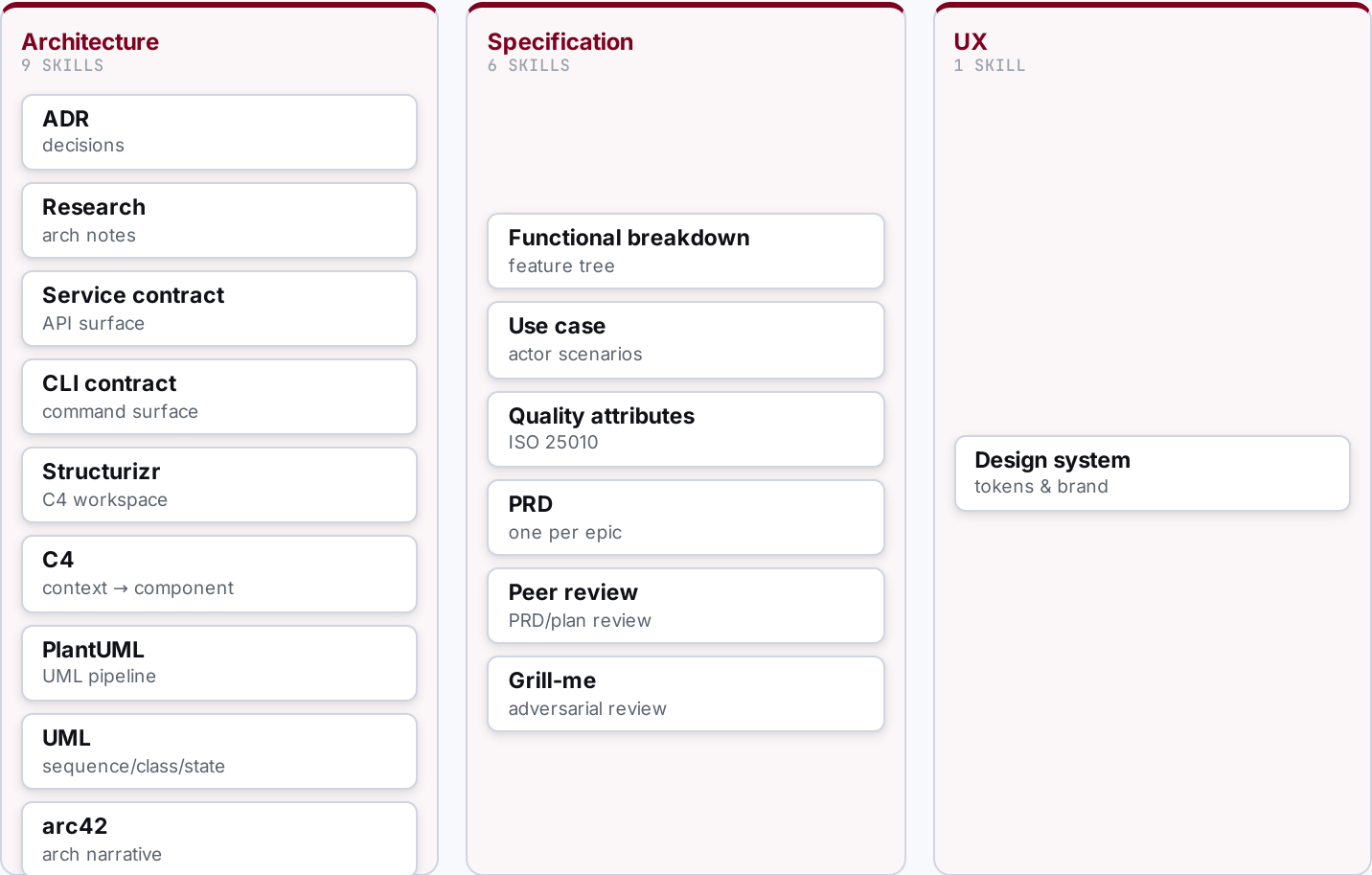
Decide architecture
ADRs, C4, arc42
- 

Define contracts
service, CLI
- 

Break down function
FBS
- 

Specify & qualify
use cases, quality
- 

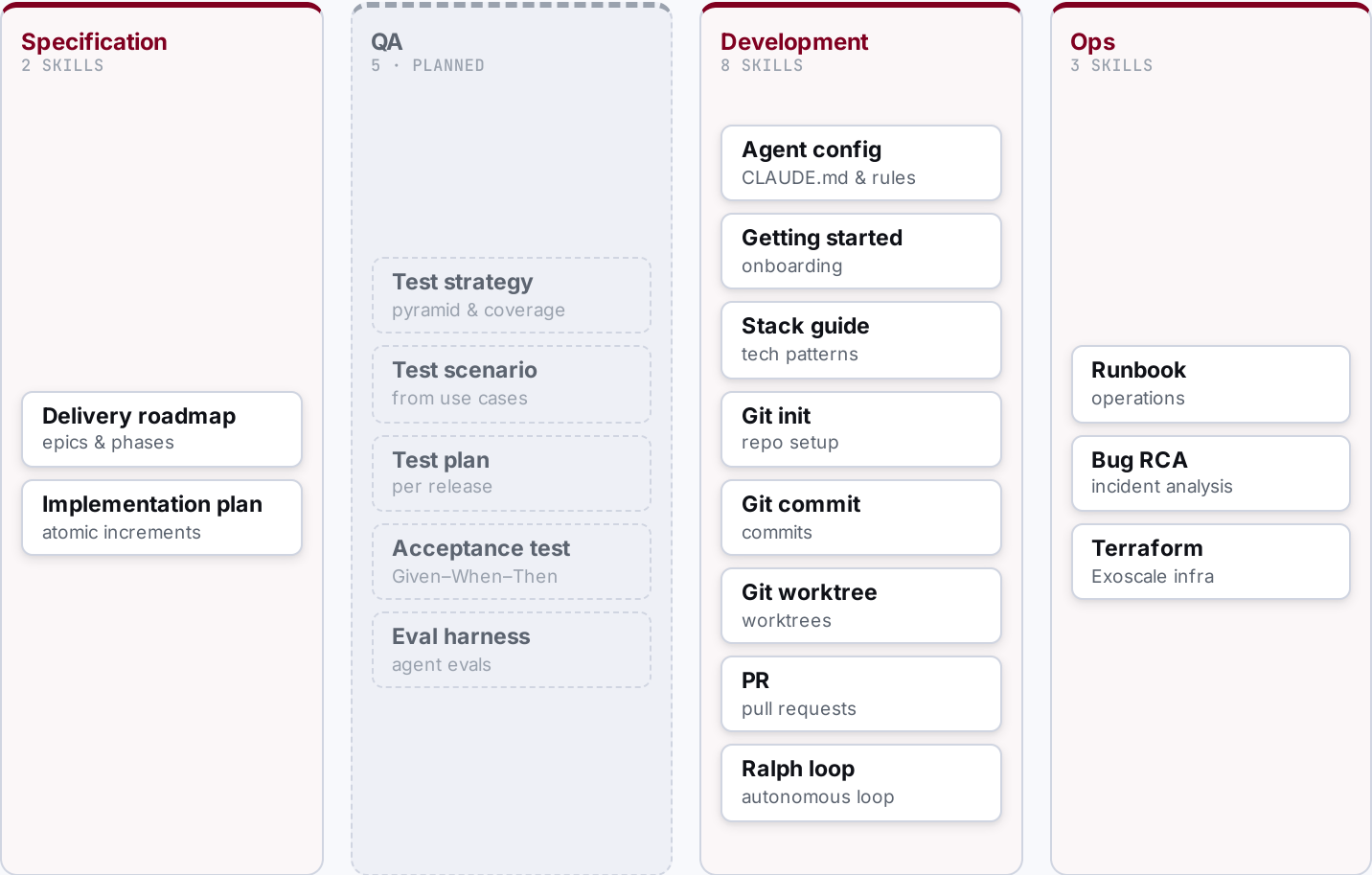
Write & review PRDs
one per epic



Execute & Deliver • plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Sequence the roadmap**
epics, phases
- **Plan increments**
atomic steps
- **Build & validate**
tests, checks
- **Deploy**
infra, release
- **Operate & fix**
runbooks, RCA



SECTION 04 / 05

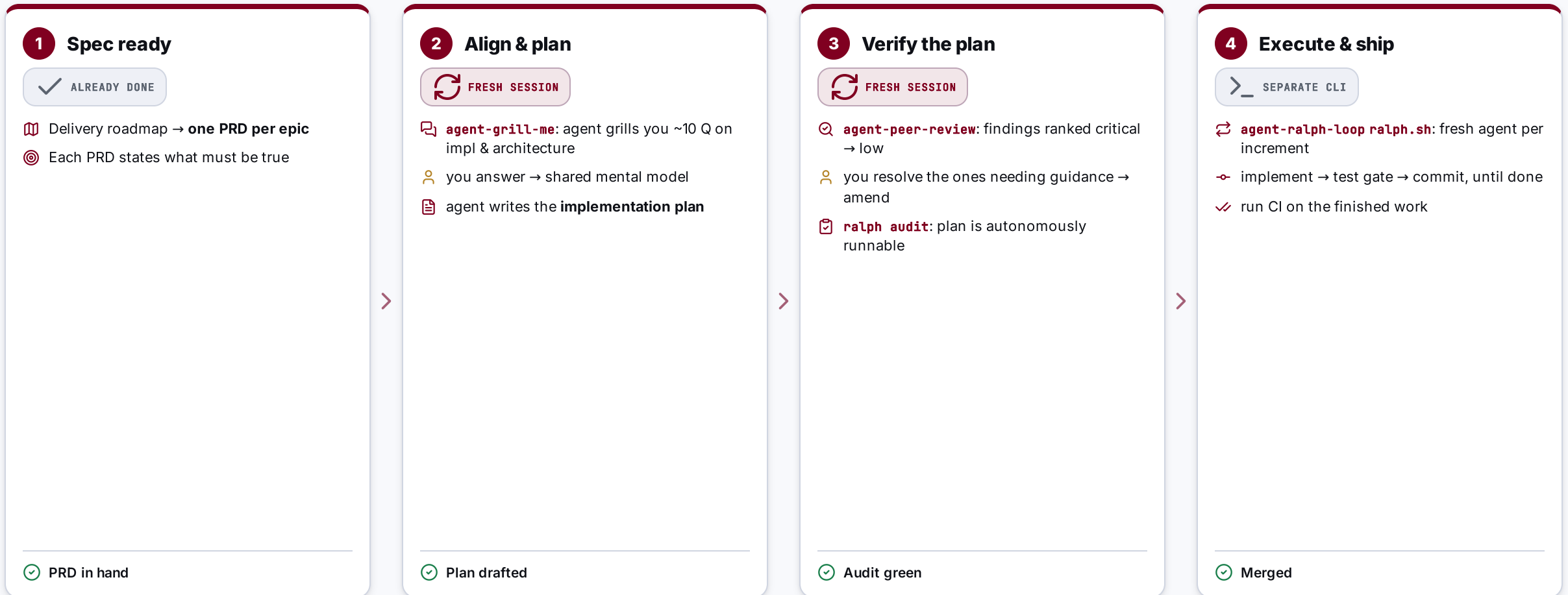
The Build **Workflow**

From a PRD to merged code: how one engineer drives agents through alignment and verification gates, then lets them run autonomously.

04

From PRD to merged code: two fresh-session gates, then autonomy.

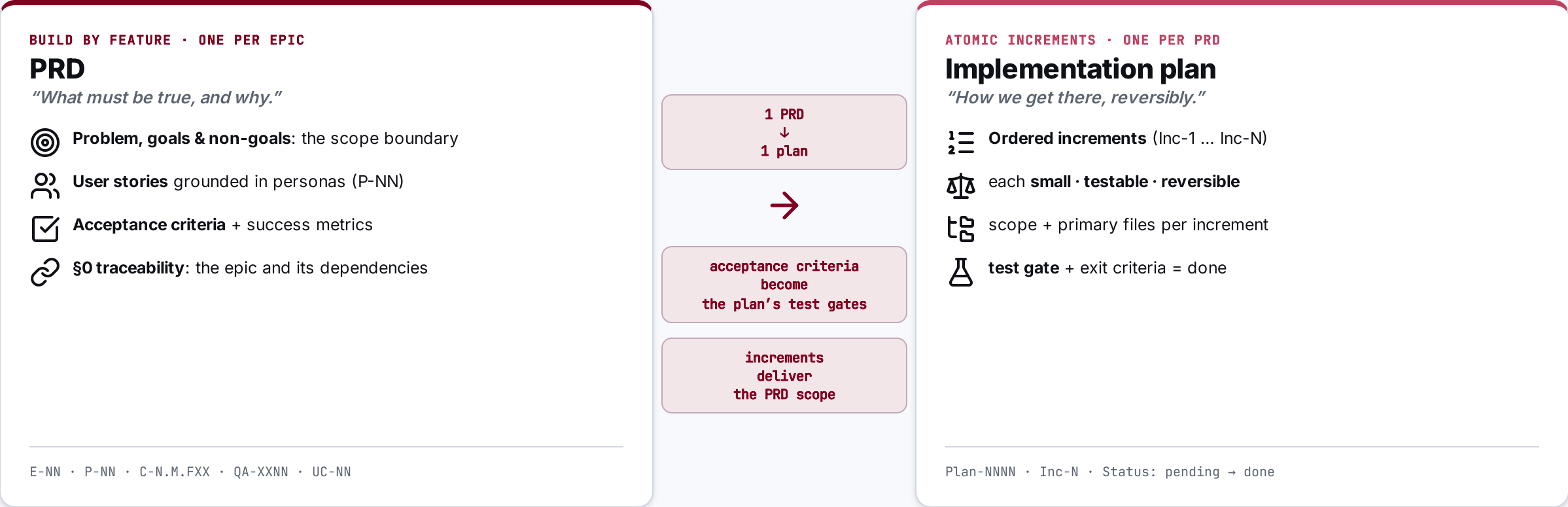
Business context and discovery are done; the roadmap gives one PRD per epic. The loop below runs per PRD.



Two **fresh-session** checks (grill-me before, peer-review after) keep me and the agent aligned; the loop only runs once the plan is proven autonomous.

PRD says what must be true; the plan says how, safely.

Two artefacts, one chain: the PRD scopes the epic, the implementation plan breaks it into runnable increments.



Two fresh agents pressure-test the work, before and after the plan.

Grill-me pulls the thinking out of my head; peer-review checks the document. Both run in a clean session.

ALIGNMENT · BEFORE THE PLAN

agent-grill-me

Live Socratic session on the PRD, one question at a time.

- 🔍 Agent asks ~10 focused Q on **implementation & architecture**
- 📖 Vocabulary checked against the **domain glossary**
- 📝 Decisions crystallised into **ADRs** while context is live
- 🧠 Surfaces the **thinking gaps** only questioning reveals

Output: **shared mental model** → the agent then writes the implementation plan.

VERIFICATION · AFTER THE PLAN

agent-peer-review

Static, independent scan of the finished plan.

- 🔍 Reads the plan end-to-end, builds a requirement map
- ⚠️ Findings ranked with concrete remediation:
 - critical
 - major
 - normal
 - low
- 🔧 I resolve the ones needing guidance → **amend the plan**

Output: **validated plan** → ready for the autonomy audit.

The Ralph loop: a fresh agent per increment, until the plan is done.

WHAT THE LOOP IS

- ↻ A bash script (**ralph.sh**) drives the plan to completion, one increment per iteration.
- ↻ **Fresh agent each iteration** → clean context, no drift or context rot.
- 🚩 Loops while increments are **pending**; stops when every one is **Status: done**.

```
ralph.sh <workspace> \  
  --max-iterations 50 --with-prd --with-push  
# spawns: claude --print < iteration-prompt
```

🔄 ONE ITERATION · PERCEIVE → IMPLEMENT → VALIDATE → COMMIT

Audit must be green before the first run.



Pick next pending increment

read workspace + plan



Implement its scope

follow the primary-files list



Run the test gate

pottery wheel, up to 3 retries



Commit the increment

advance status → done

↑ repeat with a fresh agent



All increments done → run CI

Ten commandments for the **company**.

Building with agents is an organisational capability, not a gadget. Set the conditions.

- 1 Enable the tooling.**
Give every dev agents, sandboxes and access; don't gatekeep the lever.
- 2 Standardize the workflow and artefacts, not the tool.**
Mandate the method; let people pick their own harness.
- 3 AI skips none of your real problems.**
Quality, process, planning, architecture are still there; you just hit the wall faster, and it's bigger.
- 4 Claim AI sovereignty now.**
Don't turn every model and tool into a US dependency.
- 5 Default to open source.**
OSS always ends up the default; don't repeat the Microsoft Office lock-in.

- 6 Govern the data before the agent sees it.**
Decide what company data an agent may ingest; first, not after.
- 7 Budget for maintenance.**
The harness is a living system, not a one-off.
- 8 Pay down cognitive debt.**
Don't let agents quietly erode the team's understanding.
- 9 Measure what must be true, not velocity.**
Track traceability and correctness, not tickets closed.
- 10 The method is the moat, not the model.**
Invest in the harness and the memory; models are commodities.



The model is a commodity. Your edge is the **method and memory** you build around it.

Ten commandments for the **developer**.

The agent is a power tool. The judgement, the architecture and the guardrails are still yours.

1

Build your own harness.

No one told you which IDE to use; no one tells you which agent.

2

Treat your agent like a smart junior.

Give it tools, steer it, correct it.

3

Keep a fresh context.

Start clean; don't let the window rot.

4

Agents magnify bad architecture as fast as good.

So build the guardrails.

5

The tooling is hackable.

It was built to be bent; adapt it to your workflow.

6

Start minimal; earn every tool and framework.

Hit a wall before you add a tool; run the agent loop bare before you adopt an agentic framework.

7

Fast to write is not a reason to write.

Coding was never the bottleneck; thinking was.

8

Trust, but verify.

Read the diff; every line is yours to defend.

9

Learn by playing.

Build hobby projects, not experiments on company goals.

10

Refactoring got cheap. Architecture didn't get optional.

Cheap to change is not the same as fine to wing it.



Engineer the harness, not just the prompt. The craft moved **up a rung**; it didn't disappear.

THE TAKEAWAY

Engineer the **harness**, not just the prompt.

Start with the highest-leverage capability: memory that doesn't rot.

Not a PM tool

Not a wiki

Not a SaaS

The repo is the source of truth

Git for product architecture: manage what must be true, not what's in progress.